```
"""
MEGA MCP Server
===============
FastMCP server for MEGA cloud storage using the unofficial mega.py library.

Auth env vars:
  MEGA_EMAIL     — MEGA account email
  MEGA_PASSWORD  — MEGA account password

Install deps: pip install mega.py mcp[cli] httpx pydantic
"""

import os
import json
import re
import asyncio
from typing import Optional, Dict, Any, List
from concurrent.futures import ThreadPoolExecutor
from functools import partial
from pydantic import BaseModel, Field, ConfigDict
from mcp.server.fastmcp import FastMCP

try:
    from mega import Mega
except ImportError:
    raise ImportError("Install mega.py: pip install mega.py")

# ── Constants ──────────────────────────────────────────────────────────────
MEGA_EMAIL    = os.environ.get("MEGA_EMAIL", "")
MEGA_PASSWORD = os.environ.get("MEGA_PASSWORD", "")

mcp = FastMCP("mega_mcp")
_executor = ThreadPoolExecutor(max_workers=4)

# ── Sync→Async bridge ──────────────────────────────────────────────────────
# mega.py is synchronous; run it in a thread pool to keep FastMCP non-blocking.

_mega: Optional[Mega] = None
_m_session = None  # authenticated session object


def _get_session():
    global _mega, _m_session
    if _m_session is not None:
```

```
        return _m_session
    if not MEGA_EMAIL or not MEGA_PASSWORD:
        raise RuntimeError("MEGA_EMAIL and MEGA_PASSWORD must be set.")
    _mega = Mega()
    _m_session = _mega.login(MEGA_EMAIL, MEGA_PASSWORD)
    return _m_session


async def _run(fn, *args, **kwargs):
    """Run a synchronous mega.py call in the thread executor."""
    loop = asyncio.get_event_loop()
    return await loop.run_in_executor(_executor, partial(fn, *args, **kwargs))


def _fmt_size(b: int) -> str:
    if b is None:
        return "?"
    for unit in ["B", "KB", "MB", "GB", "TB"]:
        if b < 1024:
            return f"{b:.1f} {unit}"
        b /= 1024
    return f"{b:.1f} PB"


def _fmt_node(n: Dict, key: str = "") -> str:
    t = n.get("t", 0)
    icon = "📁" if t == 1 else "📄"
    size = _fmt_size(n.get("s", 0)) if t == 0 else ""
    name = n.get("a", {}).get("n", "<unnamed>") if isinstance(n.get("a"), dict) e
    return f"{icon} **{name}**  {size}  (node=`{key}`)"


# ── Input Models ──────────────────────────────────────────────────────────
class NodeInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    node_id: str = Field(..., description="MEGA node ID (obtain from list_files)"


class FindInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    filename: str = Field(..., description="Exact or partial filename to search f


class UploadInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    local_path: str = Field(..., description="Absolute local path to upload")
    dest_dir: Optional[str] = Field(default=None, description="MEGA destination f
```

```
class DownloadInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    node_id: str = Field(..., description="MEGA node ID of file to download")
    dest_dir: str = Field(..., description="Local directory to save the file")


class CreateFolderInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    folder_name: str = Field(..., description="New folder name", min_length=1, ma
    parent_node_id: Optional[str] = Field(default=None, description="Parent node


class ShareInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    node_id: str = Field(..., description="Node ID to create a public link for")


class MoveInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    node_id: str = Field(..., description="Node ID of file or folder to move")
    dest_node_id: str = Field(..., description="Node ID of destination folder")


class RenameInput(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True, extra="forbid")
    node_id: str = Field(..., description="Node ID to rename")
    new_name: str = Field(..., description="New name")


# ── Tools ────────────────────────────────────────────────────────────────
@mcp.tool(name="mega_get_storage_info",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def mega_get_storage_info() -> str:
    """Return MEGA account storage quota and usage."""
    try:
        m = await _run(_get_session)
        quota = await _run(m.get_quota)
        storage = await _run(m.get_storage_space)
        used  = storage.get("used", 0)
        total = storage.get("total", 0)
        lines = [
            "## MEGA Storage Info",
            f"- **Used**: {_fmt_size(used)}",
            f"- **Total**: {_fmt_size(total)}",
            f"- **Free**: {_fmt_size(max(0, total - used))}",
```

```
            f"- **Quota info**: {json.dumps(quota)}",
        ]
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="mega_list_files",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def mega_list_files() -> str:
    """List all files and folders in the MEGA account (flat listing with node IDs
    try:
        m = await _run(_get_session)
        files = await _run(m.get_files)
        if not files:
            return "No files found in MEGA account."
        folders = {k: v for k, v in files.items() if v.get("t") == 1}
        file_nodes = {k: v for k, v in files.items() if v.get("t") == 0}
        lines = [f"## MEGA Files ({len(file_nodes)} files, {len(folders)} folders
        if folders:
            lines.append("### Folders")
            for k, v in list(folders.items())[:100]:
                lines.append(_fmt_node(v, k))
        if file_nodes:
            lines.append("\n### Files")
            for k, v in list(file_nodes.items())[:200]:
                lines.append(_fmt_node(v, k))
        total = len(files)
        if total > 300:
            lines.append(f"\n_Showing first 300 of {total} total items. Use mega_
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="mega_find_file",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def mega_find_file(params: FindInput) -> str:
    """Search for a file by name in MEGA (partial match)."""
    try:
        m = await _run(_get_session)
        results = await _run(m.find, params.filename)
        if not results:
            return f"No files found matching '{params.filename}'."
        lines = [f"## MEGA Search: '{params.filename}'", f"**{len(results)} found
        if isinstance(results, dict):
            for k, v in results.items():
```

```
                lines.append(_fmt_node(v, k))
        elif isinstance(results, list):
            for item in results:
                if isinstance(item, dict):
                    lines.append(f"📄 {item.get('a', {}).get('n', '?')}  node=`{i
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"


@mcp.tool(name="mega_upload_file",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def mega_upload_file(params: UploadInput) -> str:
    """Upload a local file to MEGA. Optionally specify a destination folder name.
    try:
        if not os.path.exists(params.local_path):
            return f"Error: Local file not found: {params.local_path}"
        m = await _run(_get_session)
        dest = None
        if params.dest_dir:
            dest = await _run(m.find, params.dest_dir)
            if dest is None:
                return f"Destination folder '{params.dest_dir}' not found. Use me
        size = os.path.getsize(params.local_path)
        name = os.path.basename(params.local_path)
        file_node = await _run(m.upload, params.local_path, dest)
        node_key  = file_node.get("h", file_node.get("f", [{}])[0].get("h", "unkn
        return (f"✅ Uploaded **{name}** ({_fmt_size(size)}) "
                f"to {'/' + params.dest_dir if params.dest_dir else '/'}\n"
                f"- Node ID: `{node_key}`")
    except Exception as e:
        return f"Error uploading: {e}"


@mcp.tool(name="mega_download_file",
          annotations={"readOnlyHint": True, "destructiveHint": False})
async def mega_download_file(params: DownloadInput) -> str:
    """Download a MEGA file by node ID to a local directory."""
    try:
        m = await _run(_get_session)
        files = await _run(m.get_files)
        node = files.get(params.node_id)
        if not node:
            return f"Node `{params.node_id}` not found."
        os.makedirs(params.dest_dir, exist_ok=True)
        result = await _run(m.download, node, params.dest_dir)
        name = node.get("a", {}).get("n", "?") if isinstance(node.get("a"), dict)
```

```
        return f"✅ Downloaded **{name}** to {params.dest_dir}"
    except Exception as e:
        return f"Error downloading: {e}"


@mcp.tool(name="mega_create_folder",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def mega_create_folder(params: CreateFolderInput) -> str:
    """Create a new folder in MEGA."""
    try:
        m = await _run(_get_session)
        if params.parent_node_id:
            files = await _run(m.get_files)
            parent = files.get(params.parent_node_id)
            if not parent:
                return f"Parent node `{params.parent_node_id}` not found."
            folder = await _run(m.create_folder, params.folder_name, parent)
        else:
            folder = await _run(m.create_folder, params.folder_name)
        return f"✅ Folder created: **{params.folder_name}**"
    except Exception as e:
        return f"Error creating folder: {e}"


@mcp.tool(name="mega_delete_node",
          annotations={"readOnlyHint": False, "destructiveHint": True, "idempoten
async def mega_delete_node(params: NodeInput) -> str:
    """Delete a file or folder from MEGA by node ID (moves to Trash)."""
    try:
        m = await _run(_get_session)
        files = await _run(m.get_files)
        node = files.get(params.node_id)
        if not node:
            return f"Node `{params.node_id}` not found."
        await _run(m.delete, node)
        name = node.get("a", {}).get("n", params.node_id) if isinstance(node.get(
        return f"🗑️ Deleted: **{name}** (moved to Trash)"
    except Exception as e:
        return f"Error deleting: {e}"


@mcp.tool(name="mega_export_link",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def mega_export_link(params: ShareInput) -> str:
    """Create a public shareable MEGA link for a file or folder."""
    try:
        m = await _run(_get_session)
```

```
        files = await _run(m.get_files)
        node = files.get(params.node_id)
        if not node:
            return f"Node `{params.node_id}` not found."
        link = await _run(m.export, node)
        name = node.get("a", {}).get("n", "?") if isinstance(node.get("a"), dict)
        return f"🔗 Public link for **{name}**:\n`{link}`"
    except Exception as e:
        return f"Error creating link: {e}"


@mcp.tool(name="mega_move_node",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def mega_move_node(params: MoveInput) -> str:
    """Move a MEGA file or folder to a different folder."""
    try:
        m = await _run(_get_session)
        files = await _run(m.get_files)
        node = files.get(params.node_id)
        dest = files.get(params.dest_node_id)
        if not node:
            return f"Node `{params.node_id}` not found."
        if not dest:
            return f"Destination `{params.dest_node_id}` not found."
        await _run(m.move, node, dest)
        name = node.get("a", {}).get("n", "?") if isinstance(node.get("a"), dict)
        return f"📦 Moved **{name}** to node `{params.dest_node_id}`"
    except Exception as e:
        return f"Error moving: {e}"


@mcp.tool(name="mega_rename_node",
          annotations={"readOnlyHint": False, "destructiveHint": False, "idempote
async def mega_rename_node(params: RenameInput) -> str:
    """Rename a MEGA file or folder."""
    try:
        m = await _run(_get_session)
        files = await _run(m.get_files)
        node = files.get(params.node_id)
        if not node:
            return f"Node `{params.node_id}` not found."
        await _run(m.rename, node, params.new_name)
        return f"✏️ Renamed to **{params.new_name}**"
    except Exception as e:
        return f"Error renaming: {e}"
```

```
if __name__ == "__main__":
    mcp.run()
```